

Optimization methods

1. *We are able to draw the NN*
2. *We are able to write math equations*
3. *we are able to write math equations in matrix format*
4. *we are able to calculate total parameters of any NN*
5. *we are able to calculate the parameters values*
 - *Forward propagation*
 - *Backward propagation*
 - *Epochs*
 - *Gradient descent algorithm*
- 6) *Batch GD*
- 7) *Mini batch*
- 8) *Stochastic*
- 9) *Activation functions*
- 10) *Optimization methods*
- 11) *Regularization methods (to avoid the overfit)*
- 12) *evaluation: Cross entropy*

1. Gradient Descent

- Batch GD
- MiniBatch GD
- Stochastic GD

2. Gradient descent with Momentum

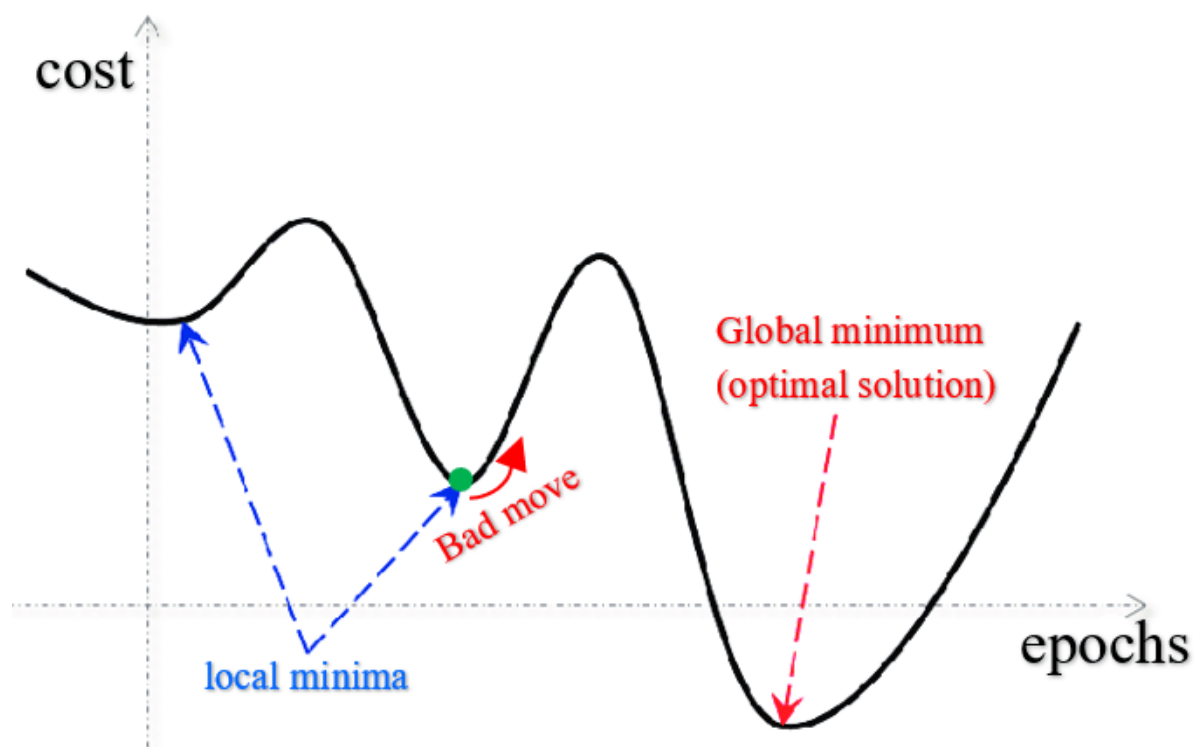
3. Adaptive Gradient Descent

4. RMS prop

5. ADAM

why Optimization methods

- optimization methods is used to find the weights and bias in more efficient way
- the main goal is to find the suitable coefficients to minimize the cost function
- but some times we have multiple minimum points will be available
 - global minima
 - local minima
- How to find the Global minima
- How to find the suitable coefficients in quick way (convergence)
- how the weights are learning based on previous iteration



local minima

- *it is a minimum point lower than the surrounding points*
- *but not a absolute minimum*
- *Gradient descent kind of optimization methods might be trap here*
- *if the optimization method find it is a minimum point then the learning will stop*
- *so it is very important to check the other minimum points exist*

Global minima

- *it is a an absolute minimum point*
- *other than this minimum point, there is no other minimum point exist*
- *Finding Global minima is always not a necessary , because no model will 100% efficient*
- *Many models works well at local minima level.*

Gradient Descent

$$w_{new} = w_{old} - \alpha * \frac{\partial J}{\partial w_{new}} |_{w_{new}=w_{old}}$$

Gradient Descent with Moomentum

we need to provide some momentum to the weights and gradients

the summation of moomentums equal to one

$$v_t = \beta * v_{t-1} + (1 - \beta) * \frac{\partial J}{\partial v_t} |_{v_t=v_{t-1}}$$

$$v_t = \beta * v_{t-1} + (1 - \beta) * \frac{\partial J}{\partial w} |_{w=w_{old}}$$

where v = velocity indicates moomentum

β = momentum value = 0.9

$$w_{new} = w_{old} - \alpha * v_t$$

Adaptive Gradient Descent(AdaGrade):

- While finding the coefficients we will use the learning rate in GD
- In GD algorithm these learning rates are fixed throughout the process
- AdaGrade we will change the learning rate for every iteration
- The learning rate is based on previous gradients

Idea: Gradients slope huge ===== Error is huge ===== Learning rate low

Gradients slope low ===== Error is low ===== Learning rate fast

$$w_{new} = w_{old} - \alpha * \frac{\partial J}{\partial w} |_{w=w_{old}} \text{ (Original GD)}$$

$$w_{new} = w_{old} - \alpha' * \frac{\partial J}{\partial w} |_{w=w_{old}} \text{ (ADA activeGD)}$$

$$\alpha' = \frac{\alpha}{\sqrt{\text{sum of squares of gradients}}}$$

$$\text{Gradient} = \frac{\partial J}{\partial w}$$

$$\text{Gradient}^2 = \left[\frac{\partial J}{\partial w} \right]^2$$

sum of squares of gradients more means the history is more
because of epochs increases parameter updation also increases
so that learning rate will be decrease

Key point: learning rate will decrease as more parameters updated
as more iterations leads to more parameter updation
and it leads to change the learning rate

assume that we have 100 parameters (weights)

so that we will get 100 gradients

$$SSG = \text{sum of square of Gradients} = \sum_{i=1}^{n=100} \left[\frac{\partial J}{\partial w_i} \right]^2$$

RMS Prop

what is the problem in Adaptive gradients descent

learning rate is change

learning rate decreasing

at some point of time learning rate will more decay or shrink

to avoid this drawback will add momentum

RMSprop = GD with Momentum + Adaptive Gradient Descent

*GD with Momentum: $E_g^2 = \beta * E_{g-1}^2 + (1 - \beta) * \frac{\partial J}{\partial w}|_{w=w_{old}}$*

*Adaptive Gradient Descent: $w_{new} = w_{old} - \alpha' * \frac{\partial J}{\partial w}|_{w=w_{old}}$*

$$\alpha' = \frac{\alpha}{\sqrt{E_g^2}}$$

$E_g^2 = \text{squared gradients}$

ADAM

Adam will add momentum for both learning rates and Cost function

Adam is very famous

Adam = Momentum + RMS prop

$$w_{new} = w_{old} - \alpha * \frac{\partial J}{\partial w}|_{w=w_{old}} \text{ (Original GD)}$$

$$w_{new} = w_{old} - \text{learning_rate} * \text{cost_function}$$

$$\alpha' = \frac{\alpha}{\sqrt{\text{sum of squares of gradients}}}$$

$$\text{cost}_{function} = j$$

$$\alpha = \beta * \alpha_{-1} + (1 - \beta) * \frac{\partial J}{\partial \mathbf{w}}|_{\mathbf{w}=\mathbf{w}_{old}}$$

$$\mathbf{j} = \beta * \mathbf{j}_{-1} + (1 - \beta) * \frac{\partial J}{\partial \mathbf{w}}|_{\mathbf{w}=\mathbf{w}_{old}}$$

the momentum will add based on weights

$$\mathbf{lr}_{new} = \beta * \mathbf{lr}_{old} + (1 - \beta) * [\mathbf{w}_{old} - \mathbf{w}_{new}]^2$$

$$\mathbf{J}_{old} = \beta * \mathbf{J}_{new} + (1 - \beta) * [\mathbf{w}_{old} - \mathbf{w}_{new}]$$

